МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине: «Безопасность систем баз данных»

по теме: «Защита БД студий дубляжа»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2026г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2026г

_____

(подпись)

Новосибирск 2026

Содержание

Введение 3
1. Реализация секционирования (партицирования) 4
2. Расчет метрик 9
3. Принятие решений и обновление данных 12
Вывод 14
Приложение А 15
Приложение Б 17
Приложение В 24

Введение

Цели работы:

1. Реализовать секционирование (партицирование) ключевых таблиц для повышения скорости аналитических запросов (обеспечение свойства Доступности).

2. Рассчитать сложные маркетинговые метрики, используя оптимизированную структуру базы данных.

3. На основе полученной аналитики выполнить модификацию схемы и данных (реализация обратной связи в бизнес-логике).

Тема: «Защита БД студий дубляжа – управление доступом к авторским материалам, контрактам с актёрами, расписаниям записей и защищённым архивам аудиофайлов».

Задачи:

1. Реализация секционирования (партицирования);

2. Расчет метрик;

3. Принятие решений и обновление данных.

1. Реализация секционирования (партицирования)

В нашей модели студии дубляжа не было модели «клиент-товар-заказ», поэтому мы создадим сущности:

Клиенты (app.customers): это клиенты, которые заказывают услуги дубляжа (Рисунок 1).

```
1  select * from app.customers;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	customer_id [PK] integer	full_name character varying (200)	email character varying (150)	phone character varying (30)	registered_at timestamp with time zone	is_active boolean
1	4	Кинокомпания «АртФильм»	artfilm@example.com	+7-900-444-5566	2024-11-20 00:00:00+00	true
2	2	Стриминговый сервис «StreamF…	streamfun@example.co…	+7-900-222-3344	2025-06-10 00:00:00+00	true
3	3	Кинотеатр «Премьер»	premiere@example.com	+7-900-333-4455	2025-09-05 00:00:00+00	true
4	1	ООО «КиноПрокат Плюс»	kinoprokat@example.c…	+7-900-111-2233	2025-01-15 00:00:00+00	true
5	5	Онлайн-кинотеатр «MovieNow»	movienow@example.co…	+7-900-555-6677	2025-12-01 00:00:00+00	true

Рисунок 1 – Таблица app.customers

Товары/услуги (app.products): дубляж фильма, серия сериала, рекламный ролик и т.д. (Рисунок 2).

```
1  select * from app.products;
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1

	product_id [PK] integer	product_name character varying (200)	category character varying (100)	base_price numeric (12,2)	is_active boolean
1	2	Дубляж серии сериала (45 минут)	Сериал	80000.00	true
2	4	Рекламный ролик до 60 секунд	Реклама	50000.00	true
3	6	Адаптация трейлера	Трейлер	40000.00	true
4	1	Полный дубляж полнометражного филь…	Фильм	300000.00	true
5	3	Локализация документального фильма	Документальный	150000.00	true
6	5	Озвучивание обучающего курса	Обучающий	120000.00	true

Рисунок 2 – Таблица app.products

Заказы (app.orders): содержит историю операций (заказов) с датами (Рисунок 3).

```
1  select * from app.orders;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10

	order_id [PK] integer	customer_id integer	order_date date	status character varying (30)	total_amount numeric (12,2)
1	1	1	2026-01-10	completed	300000.00
2	2	2	2026-01-18	completed	240000.00
3	3	3	2026-01-25	completed	100000.00
4	4	1	2026-02-03	completed	320000.00
5	5	2	2026-02-05	completed	340000.00

Рисунок 3 – Таблица app.orders

Позиции заказа (app.order_items): служит для расчёта сумм заказов и метрик по товарам (Рисунок 4).

```
1  select * from app.order_items;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10

	order_item_id [PK] integer	order_id integer	product_id integer	quantity integer	unit_price numeric (12,2)	line_total numeric (12,2)
1	1	1	1	1	300000.00	300000.00
2	2	2	2	3	80000.00	240000.00
3	3	3	4	2	50000.00	100000.00
4	4	4	1	1	320000.00	320000.00
5	5	5	2	4	85000.00	340000.00

Рисунок 4 – Таблица app.order_items

Создана новая таблица app.orders_partitioned структура которой повторяет app.orders. В объявлении таблицы указано: PARTITION BY RANGE (order_date) – секционирование по полю даты заказа (Рисунок 5).

```
1  select * from app.orders_partitioned;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10

	order_id [PK] integer	customer_id integer	order_date [PK] date	status character varying (30)	total_amount numeric (12,2)
1	1	1	2026-01-10	completed	300000.00
2	2	2	2026-01-18	completed	240000.00
3	3	3	2026-01-25	completed	100000.00
4	10	5	2025-12-15	completed	290000.00
5	4	1	2026-02-03	completed	320000.00

Рисунок 5 – Таблица app.orders_partitioned

Отчётный месяц зафиксирован на февраль 2026 года. На этой основе созданы две партиции:

app.orders_p_archive: содержит все заказы до 01.02.2026. Это соответствует архивной секции partition_archive для старых данных (Рисунок 6).

```
1  select * from app.orders_p_archive;
```

Data Output | Messages | Notifications

Showing rows: 1 to 4

	order_id [PK] integer	customer_id integer	order_date [PK] date	status character varying (30)	total_amount numeric (12,2)
1	1	1	2026-01-10	completed	300000.00
2	2	2	2026-01-18	completed	240000.00
3	3	3	2026-01-25	completed	100000.00
4	10	5	2025-12-15	completed	290000.00

Рисунок 6 – Таблица app.orders_p_archive

app.orders_p_current: содержит все заказы за февраль 2026. Это и есть секция текущего месяца partition_current для данных за последний месяц (Рисунок 7).

```
1  select * from app.orders_p_current;
```

Data Output | Messages | Notifications

Showing rows: 1 to 6

	order_id [PK] integer	customer_id integer	order_date [PK] date	status character varying (30)	total_amount numeric (12,2)
1	4	1	2026-02-03	completed	320000.00
2	5	2	2026-02-05	completed	340000.00
3	6	2	2026-02-15	completed	250000.00
4	7	3	2026-02-20	completed	155000.00
5	8	4	2026-02-22	completed	260000.00
6	9	5	2026-02-25	completed	126000.00

Рисунок 7 – Таблица app.orders_p_current

Таким образом, архив и текущий период разделены по диапазонам дат.

На рисунке 8 представлен код, который копирует все данные из таблицы orders в секционированную таблицу orders_partitioned.

```
INSERT INTO app.orders_partitioned (order_id, customer_id, order_date, status, total_amount, created_at)
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.status,
    o.total_amount,
    o.created_at
FROM app.orders o;
```

Рисунок 8 – Копирование в секционированную таблицу

Тем самым все исходные заказы из app.orders теперь присутствуют в секционированной структуре. Код реализации секционирования (партицирования) представлен в приложении А.

2. Расчет метрик

LTV – это общая выручка, которую принёс клиент за всё время взаимодействия с компанией (от первой до последней покупки). После выполнения функции получаем таблицу с каждым клиентом, его именем, суммой всех его заказов (LTV) и датами первой/последней покупки (Рисунок 9).

	client_id integer	client_name character varying (200)	ltv numeric	first_order_date date	last_order_date date
1	2	Стриминговый сервис «StreamF…	830000.00	2026-01-18	2026-02-15
2	1	ООО «КиноПрокат Плюс»	620000.00	2026-01-10	2026-02-03
3	5	Онлайн-кинотеатр «MovieNow»	416000.00	2025-12-15	2026-02-25
4	4	Кинокомпания «АртФильм»	260000.00	2026-02-22	2026-02-22
5	3	Кинотеатр «Премьер»	255000.00	2026-01-25	2026-02-20

Рисунок 9 – Вывод LTV

AOV – средняя сумма одного заказа клиента. Рассчитывается как Общая выручка клиента / Количество его заказов. После выполнения функции получаем клиентов с наибольшим средним чеком, а также информация о количестве их заказов и общей сумме (Рисунок 10).

	client_id integer	client_name character varying (200)	orders_count bigint	total_revenue numeric	aov numeric (12,2)
1	1	ООО «КиноПрокат Плюс»	2	620000.00	310000.00
2	2	Стриминговый сервис «StreamF…	3	830000.00	276666.67
3	4	Кинокомпания «АртФильм»	1	260000.00	260000.00
4	5	Онлайн-кинотеатр «MovieNow»	2	416000.00	208000.00
5	3	Кинотеатр «Премьер»	2	255000.00	127500.00

Рисунок 10 – Вывод AOV

ARPU – средний доход на одного активного клиента за период: Общая выручка за период / Количество активных клиентов. Активные клиенты – те, кто совершил хотя бы одну покупку за все время.

ARPPU – средний доход на одного платящего клиента за период: Общая выручка за период / Количество платящих клиентов в этом периоде. Платящие – те, кто совершил покупку именно в этом месяце.

В результате получаем строку с метриками за февраль: ARPU, ARPPU, количество активных и платящих клиентов, общая выручка (Рисунок 11).

	metrics text	arpu numeric	arppu numeric	active_customers_total bigint	paying_customers_month bigint	revenue_month_total numeric
1	ARPU и ARP…	725500.000000000000	870600.000000000000	6	5	4353000.00

Рисунок 11 – Вывод ARPU и ARPPU

Применим команду EXPLAIN ANALYZE к этому запросу. Увидим, что запрос просканировал только секцию текущего месяца, проигнорировав архив (Рисунок 12).

	QUERY PLAN text
1	Aggregate (cost=11.31..11.33 rows=1 width=32) (actual time=0.088..0.089 rows=1 loops=1)
2	-> Bitmap Heap Scan on orders_p_current o (cost=4.18..11.30 rows=3 width=20) (actual time=0.052..0.054 rows=6 loops=1)
3	Recheck Cond: ((order_date >= '2026-02-01'::date) AND (order_date < '2026-03-01'::date))
4	Heap Blocks: exact=1
5	-> Bitmap Index Scan on orders_p_current_pkey (cost=0.00..4.18 rows=3 width=0) (actual time=0.035..0.035 rows=6 loo…
6	Index Cond: ((order_date >= '2026-02-01'::date) AND (order_date < '2026-03-01'::date))
7	Planning Time: 0.356 ms
8	Execution Time: 0.168 ms

Рисунок 12 – EXPLAIN ANALYZE к запросу

Реализуем функцию, которая подсчитывает самые продаваемые товары по количеству проданных единиц (топ-3 популярных товаров/услуг). В результате получаем три товара, которые купили в наибольшем количестве экземпляров за февраль (Рисунок 12).

	product_id [PK] integer	product_name character varying (200)	total_quantity bigint
1	4	Рекламный ролик до 60 секунд	5
2	2	Дубляж серии сериала (45 мин...	4
3	6	Адаптация трейлера	3

Рисунок 12 – Вывод топ-3 популярных товаров/услуг

Реализуем функцию, которая подсчитывает товары с наименьшими продажами (по количеству единиц) среди тех, что вообще покупались в этом месяце. В результате получаем три товара с минимальными продажами в феврале (Рисунок 13).

	product_id [PK] integer	product_name character varying (200)	total_quantity bigint
1	1	Полный дубляж полнометражного филь...	1
2	3	Локализация документального фильма	1
3	5	Озвучивание обучающего курса	2

Рисунок 13 – Вывод топ-3 непопулярных товаров/услуг

Реализация расчета метрик представлена в приложении Б.

3. Принятие решений и обновление данных

Было принято три решение, каждое опирается на рассчитанные метрики.

1. Вводим трёхуровневую карту лояльности по суммарным тратам клиента (LTV):

- При достижении LTV больше 250 000 выдается карта лояльности со статусом Bronze с процентом скидки 5%.
- При LTV больше 400 000 устанавливается статус Silver с процентом скидки 10%.
- При LTV больше 800 000 устанавливается статус Gold с процентом скидка 15%.

В таблицу клиентов были добавлены столбцы is_vip со значением по умолчанию ложь, loyalty_level, который содержит уровень лояльности клиента и loyalty_discount_percent, который содержит процент скидки.

Затем для каждого клиента был рассчитан его LTV. После этого у каждого клиента обновляется VIP-статус (присваивается true или false), если у клиента VIP-статус равен true, ему выдается карта лояльности с начальным уровнем BRONZE (процент скидки 5%). Обновленная таблица клиентов представлена на рисунке 13.

	customer_id [PK] integer	full_name character varying (200)	email charact	phone characte	register timestal	is_active boolean	is_vip boolean	loyalty_level text	loyalty_discount_percent numeric (5,2)
1	4	Кинокомпания «АртФильм»	artfi…	+7-9…	202…	true	true	BRONZE	5.00
2	2	Стриминговый сервис «StreamF…	stre…	+7-9…	202…	true	true	GOLD	15.00
3	3	Кинотеатр «Премьер»	pre…	+7-9…	202…	true	true	BRONZE	5.00
4	1	ООО «КиноПрокат Плюс»	kino…	+7-9…	202…	true	true	SILVER	10.00
5	5	Онлайн-кинотеатр «MovieNow»	mov…	+7-9…	202…	true	true	SILVER	10.00

Рисунок 13 – Обновленная таблица app.customers

Пример запроса на покупку товара привилегированного клиента (Рисунок 14).

	customer_name text	is_vip boolean	product_name text	quantity integer	base_price numeric	product numeric	price_after_product_disc numeric	vip_loyalty_pct numeric	unit_price_after_vip numeric
1	ООО «КиноПрокат Плю…	true	Полный дубляж по…	2	300000.00	20.00	240000.00	10.00	216000.00

Рисунок 14 – Запрос на покупку товара

2. На основе частоты покупок товаров за отчётный месяц выбираем товары, которые покупают редко и назначаем скидку.

В таблицу товаров были добавлены столбец discount_percent, который содержит процент скидки на товар и столбец final_price, в котором указана цена после применения скидки, без VIP.

Затем считаем, сколько единиц каждого товара продали за месяц, выбираем топ 3 товаров с наименьшим количеством покупок (среди тех, что хотя бы раз были куплены), этим товарам назначаем скидку 20% в поле discount_percent. Обновленная таблица товаров представлена на рисунке 15.

	product_id [PK] integer	product_name character varying (200)	category character varying (100)	base_price numeric (12,2)	is_active boolean	created timesta	discount_percent numeric (5,2)	final_price numeric (12,2)
1	2	Дубляж серии сери…	Сериал	80000.00	true	202…	0.00	80000.00
2	4	Рекламный ролик д…	Реклама	50000.00	true	202…	0.00	50000.00
3	6	Адаптация трейлера	Трейлер	40000.00	true	202…	0.00	40000.00
4	1	Полный дубляж по…	Фильм	300000.00	true	202…	20.00	240000.00
5	3	Локализация доку…	Документальный	150000.00	true	202…	20.00	120000.00
6	5	Озвучивание обуча…	Обучающий	120000.00	true	202…	20.00	96000.00

Рисунок 15 – Обновленная таблица app.products

3. Корректировка скидок по карте лояльности на основе ARPPU.

После того как клиентам уже выставлены уровни лояльности и процент скидки (решение 1), мы смотрим на ARPPU за отчётный месяц: сколько в среднем выручки пришлось на одного платящего клиента в этом месяце. Если этот показатель ниже 280 000, считаем, что прибыль проседает, и усиливаем программу лояльности всем, у кого уже есть карта, увеличиваем скидку по карте на 5 процентов (к уже действующему проценту скидки по карте). Обновленная таблица клиентов представлена на рисунке 16.

	customer_id [PK] integer	full_name character varying (2	email charact	phone charact	register timesta	is_active boolean	is_vip boolean	loyalty_level text	loyalty_discount_percent numeric (5,2)	arppu_bonus_percent numeric (5,2)
1	4	Кинокомпания…	artfi…	+7-9…	202…	true	true	BRONZE	5.00	0.00
2	2	Стриминговый…	stre…	+7-9…	202…	true	true	GOLD	15.00	0.00
3	3	Кинотеатр «Пр…	pre…	+7-9…	202…	true	true	BRONZE	5.00	0.00
4	1	ООО «КиноПро…	kino…	+7-9…	202…	true	true	SILVER	10.00	0.00
5	5	Онлайн-киноте…	mov…	+7-9…	202…	true	true	SILVER	10.00	0.00

Рисунок 16 – Обновленная таблица app.customers

Реализация обновления данных представлена в приложении В.

Вывод

В данной лабораторной работе по предметной области студий дубляжа была реализована база данных с реализованным секционированием, расчетом метрик и принятие трех решений для увеличения прибыли.

На первом этапе мы реализовали секционирование таблицы заказов по диапазону дат. Была создана таблица orders_partitioned с разделением на архивную партицию для всех заказов до отчетного месяца и текущую партицию для заказов отчетного месяца.

На втором этапе были рассчитаны основные маркетинговые метрики: для каждого клиента определена пожизненная ценность LTV, на основе которой выявлены наиболее ценные клиенты; вычислен средний чек AOV; для отчетного месяца рассчитаны средний доход на активного клиента ARPU и средний доход на платящего клиента ARPPU, при этом с помощью EXPLAIN ANALYZE убедились, что запросы сканируют только текущую партицию, игнорируя архив; также были определены топ-3 популярных и топ-3 непопулярных товаров за отчётный месяц по количеству проданных единиц.

На третьем этапе на основе рассчитанных метрик были реализованы следующие изменения структуры: была добавлена карта лояльности с тремя уровнями на основе LTV; для трёх наименее популярных товаров назначена скидка двадцать процентов; если показатель ARPPU ниже порога, то добавляем скидку 5 процентов в карте лояльности.

Приложение А

```
-- Задание 1. Реализация секционирования (партицирования)

SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

-- Создание новой секционированной таблицы

DROP TABLE IF EXISTS app.orders_partitioned CASCADE;

CREATE TABLE app.orders_partitioned (
   order_id       INTEGER       NOT NULL,
   customer_id    INTEGER       NOT NULL,
   order_date     DATE          NOT NULL,
   status         VARCHAR(30),
   total_amount   NUMERIC(12,2) NOT NULL,
   created_at     TIMESTAMPTZ   DEFAULT NOW(),
   CONSTRAINT orders_partitioned_pk PRIMARY KEY (order_date, order_id)
)
PARTITION BY RANGE (order_date);

-- Создание партиций (секций)

DROP TABLE IF EXISTS app.orders_p_archive;
DROP TABLE IF EXISTS app.orders_p_current;

CREATE TABLE app.orders_p_archive
   PARTITION OF app.orders_partitioned
   FOR VALUES FROM (MINVALUE) TO ('2026-02-01');
```

```
CREATE TABLE app.orders_p_current
    PARTITION OF app.orders_partitioned
    FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');

-- Перенос данных из основной таблицы app.orders в секционированную

INSERT INTO app.orders_partitioned (order_id, customer_id, order_date, status,
total_amount, created_at)
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.status,
    o.total_amount,
    o.created_at
FROM app.orders o;
RESET ROLE;
```

Приложение Б

```
-- Задание 2. Расчет метрик

SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

-- 1. LTV - общая выручка от каждого клиента за весь период

WITH customer_orders AS (
    SELECT
        c.customer_id,
        c.full_name,
        MIN(o.order_date) AS first_order_date,
        MAX(o.order_date) AS last_order_date,
        SUM(o.total_amount) AS total_revenue
    FROM app.customers c
    JOIN app.orders_partitioned o ON o.customer_id = c.customer_id
    GROUP BY c.customer_id, c.full_name
),

customer_ltv AS (
    SELECT
        customer_id,
        full_name,
        total_revenue AS ltv,
        first_order_date,
        last_order_date
    FROM customer_orders
)
```

```
SELECT
    customer_id AS client_id,
    full_name AS client_name,
    ltv,
    first_order_date,
    last_order_date
FROM customer_ltv
ORDER BY ltv DESC;

-- 2. AOV - средний чек клиента

WITH customer_stats AS (
    SELECT
        c.customer_id,
        c.full_name,
        COUNT(o.order_id) AS orders_count,
        SUM(o.total_amount) AS total_revenue
    FROM app.customers c
    JOIN app.orders_partitioned o ON o.customer_id = c.customer_id
    GROUP BY c.customer_id, c.full_name
),

customer_aov AS (
    SELECT
        customer_id,
        full_name,
        total_revenue,
        orders_count,
        (total_revenue / NULLIF(orders_count, 0))::NUMERIC(12,2) AS aov
    FROM customer_stats
```

```
)
SELECT
    customer_id AS client_id,
    full_name AS client_name,
    orders_count,
    total_revenue,
    aov
FROM customer_aov
ORDER BY aov DESC

-- Блок Б. Метрики за отчетный период

WITH params AS (SELECT DATE '2026-02-01' AS period_start, DATE '2026-
03-01' AS period_end),

orders_month AS (
    SELECT o.*
    FROM app.orders_partitioned o
    CROSS JOIN params p
    WHERE o.order_date >= p.period_start AND o.order_date <  p.period_end),

active_customers AS (
    -- Активные клиенты: хотя бы один заказ за все время
    SELECT DISTINCT o.customer_id
    FROM app.orders_partitioned o),

paying_customers_month AS (
    -- Платящие клиенты за отчетный месяц
    SELECT DISTINCT om.customer_id
    FROM orders_month om),
```

```
revenue_month AS (
    SELECT SUM(om.total_amount) AS revenue
    FROM orders_month om)

SELECT 'ARPU и ARPPU' AS metrics,
    (SELECT revenue FROM revenue_month) / NULLIF((SELECT COUNT(*)
FROM active_customers), 0)::NUMERIC(12,2) AS arpu,
    (SELECT revenue FROM revenue_month) / NULLIF((SELECT COUNT(*)
FROM paying_customers_month), 0)::NUMERIC(12,2) AS arppu,
    (SELECT COUNT(*) FROM active_customers) AS active_customers_total,
    (SELECT COUNT(*) FROM paying_customers_month) AS
paying_customers_month,
    (SELECT revenue FROM revenue_month) AS revenue_month_total;

-- 5. Топ-3 популярных товаров/услуг за последний месяц

WITH params AS (
    SELECT
        DATE '2026-02-01' AS period_start,
        DATE '2026-03-01' AS period_end
),

orders_month AS (
    SELECT o.order_id
    FROM app.orders_partitioned o
    CROSS JOIN params p
    WHERE o.order_date >= p.period_start AND o.order_date <  p.period_end
),
```

```
items_month AS (
    SELECT oi.product_id, oi.quantity
    FROM app.order_items oi
    JOIN orders_month omON oi.order_id = om.order_id
),

product_stats AS (
    SELECT
        p.product_id,
        p.product_name,
        SUM(im.quantity) AS total_quantity
    FROM app.products p
    JOIN items_month im ON im.product_id = p.product_id
    GROUP BY p.product_id, p.product_name
)

SELECT
    product_id,
    product_name,
    total_quantity
FROM product_stats
ORDER BY total_quantity DESC
LIMIT 3;

-- 6. Топ-3 непопулярных товаров/услуг за последний месяц (среди тех, что покупались)

WITH params AS (
    SELECT
        DATE '2026-02-01' AS period_start,
```

```
        DATE '2026-03-01' AS period_end
),

orders_month AS (
    SELECT o.order_id
    FROM app.orders_partitioned o
    CROSS JOIN params p
    WHERE o.order_date >= p.period_start AND o.order_date <  p.period_end
),

items_month AS (
    SELECT oi.product_id, oi.quantity
    FROM app.order_items oi
    JOIN orders_month om ON oi.order_id = om.order_id
),

product_stats AS (
    SELECT
        p.product_id,
        p.product_name,
        SUM(im.quantity) AS total_quantity
    FROM app.products p
    JOIN items_month im ON im.product_id = p.product_id
    GROUP BY p.product_id, p.product_name
)

SELECT
    product_id,
    product_name,
    total_quantity
```

```
FROM product_stats
ORDER BY total_quantity ASC
LIMIT 3;

RESET ROLE;
```

Приложение В

```
-- Задание 3. Принятие решений и обновление данных на основе метрик

SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

-- 1. Уровни лояльности основе LTV

ALTER TABLE IF EXISTS app.customers
   ADD COLUMN IF NOT EXISTS is_vip BOOLEAN DEFAULT FALSE,
   ADD COLUMN IF NOT EXISTS loyalty_level TEXT,
   ADD COLUMN IF NOT EXISTS loyalty_discount_percent NUMERIC(5,2)
DEFAULT 0;

WITH customer_orders AS (
   SELECT
      c.customer_id,
      c.full_name,
      SUM(o.total_amount) AS total_revenue
   FROM app.customers c
   JOIN app.orders_partitioned o ON o.customer_id = c.customer_id
   GROUP BY c.customer_id, c.full_name
)

UPDATE app.customers c
SET loyalty_level = CASE
      WHEN co.total_revenue >= 800000 THEN 'GOLD'
      WHEN co.total_revenue >= 400000 THEN 'SILVER'
      WHEN co.total_revenue >= 250000 THEN 'BRONZE'
```

```
        ELSE NULL
    END,

    loyalty_discount_percent = CASE
        WHEN co.total_revenue >= 800000 THEN 15.00
        WHEN co.total_revenue >= 400000 THEN 10.00
        WHEN co.total_revenue >= 250000 THEN 5.00
        ELSE 0.00
    END,
    is_vip = (co.total_revenue >= 250000)
FROM customer_orders co
WHERE c.customer_id = co.customer_id;

-- 2. Назначение скидки к товарам на основе их непопулярности

ALTER TABLE IF EXISTS app.products
    ADD COLUMN IF NOT EXISTS discount_percent NUMERIC(5,2) DEFAULT
0;

ALTER TABLE IF EXISTS app.products
    ADD COLUMN IF NOT EXISTS final_price NUMERIC(12,2);

WITH params AS (
    SELECT
        DATE '2026-02-01' AS period_start,
        DATE '2026-03-01' AS period_end
),
```

```
orders_month AS (
    SELECT o.order_id
    FROM app.orders_partitioned o
    CROSS JOIN params p
    WHERE o.order_date >= p.period_start
    AND o.order_date <  p.period_end
),

items_month AS (
    SELECT oi.product_id, oi.quantity
    FROM app.order_items oi
    JOIN orders_month om ON oi.order_id = om.order_id
),

product_stats AS (
    SELECT
        p.product_id,
        SUM(im.quantity) AS total_quantity
    FROM app.products p
    JOIN items_month im ON im.product_id = p.product_id
    GROUP BY p.product_id)

-- Топ-3 наименее популярных товаров среди тех, что покупались за отчётный месяц

    SELECT ps.product_id
    FROM product_stats ps
    ORDER BY ps.total_quantity ASC
    LIMIT 3
)
```

```
UPDATE app.products p
SET discount_percent = 20.00
FROM least_popular lp
WHERE p.product_id = lp.product_id;

-- Конечная цена товара после скидки на товар
UPDATE app.products
SET final_price = ROUND((base_price * (100 - COALESCE(discount_percent,
0)) / 100)::NUMERIC, 2);

-- 3. Корректировка скидок по карте лояльности по ARPPU за отчётный месяц

ALTER TABLE IF EXISTS app.customers
   ADD COLUMN IF NOT EXISTS arppu_bonus_percent NUMERIC(5,2)
DEFAULT 0;

DO $$
DECLARE
   v_arppu NUMERIC(18,2);
   v_bonus INTEGER;
BEGIN
   WITH params AS (
      SELECT DATE '2026-02-01' AS d1, DATE '2026-03-01' AS d2
   ),
   om AS (
      SELECT o.total_amount, o.customer_id
      FROM app.orders_partitioned o
      CROSS JOIN params p
      WHERE o.order_date >= p.d1 AND o.order_date < p.d2
   )
```

```
    SELECT
        ROUND((COALESCE(SUM(om.total_amount), 0) /
NULLIF(COUNT(DISTINCT om.customer_id), 0))::NUMERIC, 2)
    INTO v_arppu
    FROM om;

    IF v_arppu IS NULL OR v_arppu >= 280000 THEN
        UPDATE app.customers
        SET arppu_bonus_percent = 0
        WHERE loyalty_level IS NOT NULL;
    ELSE
        UPDATE app.customers
        SET loyalty_discount_percent = loyalty_discount_percent + 5.00,
            arppu_bonus_percent = 5.00
        WHERE loyalty_level IS NOT NULL;
    END IF;
END;
$$;

RESET ROLE;
```